

# Artificial Intelligence & Expert Systems Lab

**Nahida Marzan**

Lecturer

Department of CSE, UAP

Submitted by:

22101040: Shahriar Arefin

22101043: Umme Kulsum Surovi

Section: A



# Problem Statement

The problem is to implement the classic game of Tic Tac Toe as an adversarial search problem. The goal is to design an AI player capable of making optimal moves by applying the Minimax algorithm with Alpha-Beta pruning. The AI should evaluate possible game states to choose the best action and ensure optimal play against either a human or another AI. Additionally, the efficiency of the algorithm and the effect of scoring on AI behavior need to be analyzed.

## Approach and Implementation

### 1. Game Implementation

- A 3x3 Tic Tac Toe board was implemented using a Python list to represent the state of the grid.
- Console output was used to print the board after each move.

### 2. Adversarial Search (Minimax)

- The Minimax algorithm was implemented to recursively evaluate all possible game states.
- The maximizing player (X) aims to maximize the score, while the minimizing player (O) tries to minimize it.
- Terminal states were assigned scores: +1 (X wins), -1 (O wins), 0 (draw).

### 3. Alpha-Beta Pruning

- $\alpha$  (best guaranteed score for maximizer) and  $\beta$  (best guaranteed score for minimizer) were applied to prune branches where  $\beta \leq \alpha$ .
- This reduced the number of explored nodes, which makes the algorithm more efficient.

### 4. Experiment Setup

- Multiple test board states were used to compare outcomes.
- Node exploration counts were tracked to measure efficiency.

**Source code:** The complete source code for this project is available on [GitHub](#).

# Win Combinations

Rows

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

Columns

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

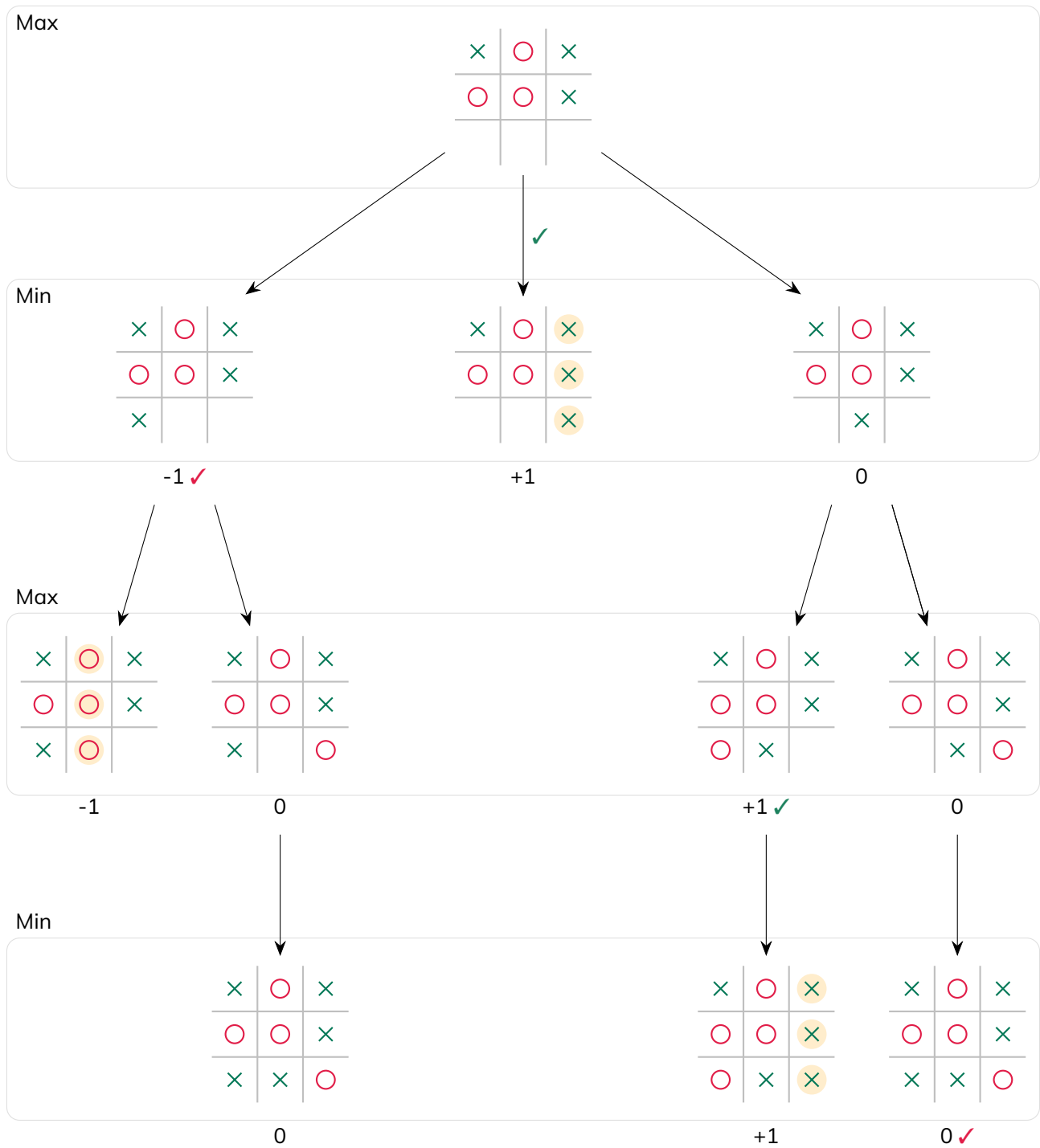
|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

Diagonals

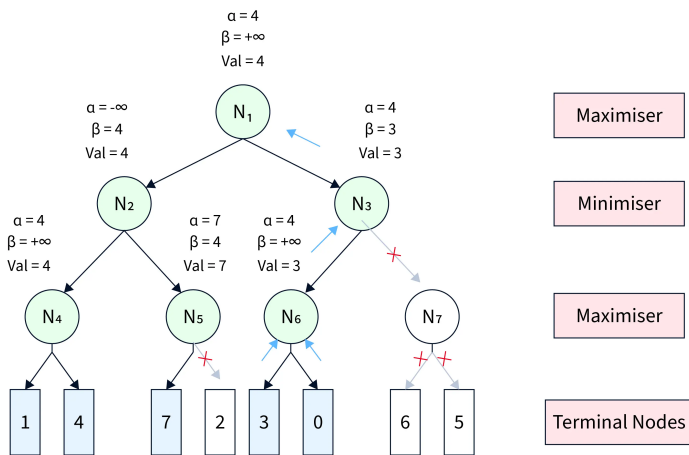
|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

# Simulation of the MinMax Algorithm



# Alpha Beta Pruning



- Initialize  $\alpha = -\infty$  and  $\beta = +\infty$  best value found so far.
- Traverse the game tree using DFS.
- At each maximizer node:
  - Update  $\alpha$  if a better max value is found.
  - Prune remaining children if  $\beta \leq \alpha$ .
- At each minimizer node:
  - Update  $\beta$  if a better min value is found.
  - Prune remaining children if  $\beta \leq \alpha$ .
- Return the static evaluation (leaf value).

## Gameplay: Human vs Human

Below is a sample run of a complete Tic-Tac-Toe game between two human players. The board is displayed after each move, using a monospaced font for clarity.

### Board State (after each move)

```

  |  |
--+--+--
  |  |
--+--+--
  |  |

```

```

X |  |
--+--+--
  |  |
--+--+--
  |  |

```

```

X |  |
--+--+--
  | 0 |
--+--+--
  |  |

```

```

X |  |
--+--+--
  | 0 |
--+--+--
  |  | X

```

```

X |  | 0
--+--+--
  | 0 |
--+--+--
  |  | X

```

```

X |  | 0
--+--+--
  | 0 |
--+--+--
X |  | X

```

```

X | 0 | 0
--+--+--
  | 0 |
--+--+--
X |  | X

```

```

X | 0 | 0
--+--+--
  | 0 |
--+--+--
X | X | X

```

**X wins!**

# Gameplay: AI vs AI

Below is a sample run of a complete Tic-Tac-Toe game between two AI players. The board is displayed after each move, using a monospaced font for clarity.

## Board State (after each move)

|           |           |           |
|-----------|-----------|-----------|
| X         | X         | X     X   |
| --+---+-- | --+---+-- | --+---+-- |
|           | 0         | 0         |
| --+---+-- | --+---+-- | --+---+-- |
|           |           |           |
|           |           |           |
| X   0   X | X   0   X | X   0   X |
| --+---+-- | --+---+-- | --+---+-- |
| 0         | 0         | 0   0     |
| --+---+-- | --+---+-- | --+---+-- |
|           | X         | X         |
|           |           |           |
| X   0   X | X   0   X | X   0   X |
| --+---+-- | --+---+-- | --+---+-- |
| 0   0   X | 0   0   X | 0   0   X |
| --+---+-- | --+---+-- | --+---+-- |
| X         | X   0     | X   X   0 |

**It's a draw!**

**Note:** This example demonstrates a full game between two AI agents, both playing optimally. The board is shown after each move, and the game ends in a draw, as expected from perfect play on both sides.

## Results and Observations

- In all evaluated test cases, the AI consistently identified the optimal move.
- Under conditions of optimal play from both participants, the game invariably concluded in a draw, thereby validating the correctness of the implementation.
- Furthermore, the incorporation of Alpha-Beta pruning led to a substantial reduction in the number of nodes explored relative to the standard Minimax algorithm.

## Challenges

- **Correct pruning logic:** ensuring  $\alpha$  and  $\beta$  updates were implemented properly.
- **Move ordering:** efficiency of pruning varied depending on exploration order.
- **Tie-breaking:** In scenarios where multiple moves were evaluated as equally optimal, the algorithm picked the first move encountered during its search process.

## Conclusion

The implementation of **Minimax with Alpha-Beta pruning** successfully produced an optimal Tic Tac Toe AI. The algorithm ensures optimal play and never loses against any opponent. Alpha-Beta pruning greatly reduces computation without affecting correctness.

For a simple game like Tic Tac Toe, the approach works perfectly and demonstrates the strength of adversarial search. This experiment confirms that adversarial search is a powerful technique for two-player zero-sum games and forms the foundation for more complex AI systems in games such as Chess or Go.